

final

December 11, 2025

1 initialize model

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# import lightgbm as lgb

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)

[2]: df = pd.read_csv('data.csv')
df.rename(columns={ df.columns[0]: "date" }, inplace = True)
```

2 XGBoost

```
[3]: base_features = [
    '^VIX',
    '^GSPC',
    'CCC_10Y_Spread',
    'Unemployment',
    'Inflation',
    'GDP_Growth',
    'Corporate_Leverage',
    'YieldCurveSlope',
    'IndPro_Growth',
    'CP_Growth',
]
```

```

# Assume df is already loaded externally
df = df.sort_values("date")

# Convenience alias
df["spread"] = df["CCC_10Y_Spread"]

# Directional features on spread (using levels)
df["spread_ret_30"] = df["spread"].diff(30)
df["spread_ret_60"] = df["spread"].diff(60)
df["spread_ret_180"] = df["spread"].diff(180)
df["spread_ret_360"] = df["spread"].diff(365)

df["spread_vol_30"] = df["spread"].diff().rolling(30).std()
df["spread_vol_60"] = df["spread"].diff().rolling(60).std()
df["spread_vol_180"] = df["spread"].diff().rolling(180).std()
df["spread_vol_360"] = df["spread"].diff().rolling(365).std()

# First-difference (change) of base macro features
for col in base_features:
    df[col + "_week"] = df[col].diff(7)
    df[col + "_month"] = df[col].diff(30)
    df[col + "_half_y"] = df[col].diff(180)
    df[col + "_year"] = df[col].diff(365)

# Drop rows with NaNs from shifting/diff/rolling
df = df.dropna().copy()

train_end = '2016-12-31'
val_end = '2021-12-31'

train = df[df['date'] <= train_end]
val = df[(df['date'] > train_end) & (df['date'] <= val_end)]
test = df[df['date'] > val_end]

# Numeric features: original + engineered
feature_cols_num = (
    base_features +
    [ "spread_ret_30", "spread_ret_60",
      "spread_ret_180", "spread_ret_360",
      "spread_vol_30", "spread_vol_60",
      "spread_vol_180", "spread_vol_360",]
    + [col + "_week" for col in base_features] +
    [col + "_month" for col in base_features] +
    [col + "_half_y" for col in base_features] +

```

```

        [col + "_year" for col in base_features]
    )

    # If you later add categoricals, put names here
    feature_cols_cat = [] # e.g. ['rating', 'sector']

    X_train = train[feature_cols_num]
    X_val    = val[feature_cols_num]
    X_test   = test[feature_cols_num]

    X_train_lgb = X_train
    X_val_lgb   = X_val
    X_test_lgb  = X_test

    # Target is the spread LEVEL
    y_train = train["spread"]
    y_val    = val["spread"]
    y_test   = test["spread"]

    preprocess = ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), feature_cols_num),
            ('cat', OneHotEncoder(handle_unknown='ignore'), feature_cols_cat),
        ],
        remainder='drop'
    )

    preprocess.fit(X_train)

    X_train_t = preprocess.transform(X_train)
    X_val_t    = preprocess.transform(X_val)
    X_test_t   = preprocess.transform(X_test)

```

```

[4]: xgb_model = XGBRegressor(
    n_estimators=5000,
    max_depth=8,
    learning_rate=0.01,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=5,
    reg_lambda=2.0,
    reg_alpha=1.0,
    objective='reg:squarederror',
    random_state=42,

```

```
)

xgb_model.fit(
    X_train_t, y_train,
    eval_set=[(X_val_t, y_val)],
    verbose=False
)
```

```
[4]: XGBRegressor(base_score=None, booster=None, callbacks=None,
                  colsample_bylevel=None, colsample_bynode=None,
                  colsample_bytree=0.8, device=None, early_stopping_rounds=None,
                  enable_categorical=False, eval_metric=None, feature_types=None,
                  feature_weights=None, gamma=None, grow_policy=None,
                  importance_type=None, interaction_constraints=None,
                  learning_rate=0.01, max_bin=None, max_cat_threshold=None,
                  max_cat_to_onehot=None, max_delta_step=None, max_depth=8,
                  max_leaves=None, min_child_weight=5, missing=nan,
                  monotone_constraints=None, multi_strategy=None, n_estimators=5000,
                  n_jobs=None, num_parallel_tree=None, ...)
```

```
[5]: lgbm_model = LGBMRegressor(
    n_estimators=3000,
    learning_rate=0.02,
    num_leaves=31,
    max_depth=-1,           # let num_leaves control tree size
    subsample=0.8,
    colsample_bytree=0.8,
    objective='regression',
    random_state=42,
    # if your version accepts n_jobs, you can add: n_jobs=-1
)

lgbm_model.fit(
    X_train_lgb, y_train,
    eval_set=[(X_val_lgb, y_val)],
    eval_metric='rmse',
)
```

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
testing was 0.001668 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 14763
[LightGBM] [Info] Number of data points in the train set: 6941, number of used
features: 58
[LightGBM] [Info] Start training from score 0.114654
```

```
[5]: LGBMRegressor(colsample_bytree=0.8, learning_rate=0.02, n_estimators=3000,
                  objective='regression', random_state=42, subsample=0.8)
```

3 Metrics

```
[6]: def metrics_with_direction(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    # MAPE on levels
    denom = np.where(y_true == 0, 1e-8, y_true)
    mape = np.mean(np.abs((y_true - y_pred) / denom)) * 100

    # Directional Accuracy based on level changes
    true_dir = np.sign(y_true[1:] - y_true[:-1])
    pred_dir = np.sign(y_pred[1:] - y_pred[:-1])
    da = np.mean(true_dir == pred_dir)

    return mae, rmse, r2, mape, da

# y_pred_train = lgbm_model.predict(X_train)
# y_pred_val    = lgbm_model.predict(X_val)

[7]: y_pred_train_x = xgb_model.predict(X_train_t)
    y_pred_val_x    = xgb_model.predict(X_val_t)

    train_mae, train_rmse, train_r2, train_mape, train_da = metrics_with_direction(
        y_train.values, y_pred_train_x
    )
    val_mae, val_rmse, val_r2, val_mape, val_da = metrics_with_direction(
        y_val.values, y_pred_val_x
    )

    print("=== TRAIN METRICS (level) ===")
    print(f"MAE : {train_mae:.4f}")
    print(f"RMSE: {train_rmse:.6f}")
    print(f"R2 : {train_r2:.4f}")
    print(f"MAPE: {train_mape:.2f}%")
    print(f"DA : {train_da:.4f}")

    print("\n=== VALIDATION METRICS (level) ===")
    print(f"MAE : {val_mae:.4f}")
    print(f"RMSE: {val_rmse:.6f}")
    print(f"R2 : {val_r2:.4f}")
    print(f"MAPE: {val_mape:.2f}%")
```

```
print(f"DA : {val_da:.4f}")
```

```
=== TRAIN METRICS (level) ===
```

```
MAE : 0.0010  
RMSE: 0.002915  
R2 : 0.9974  
MAPE: 0.98%  
DA : 0.7524
```

```
=== VALIDATION METRICS (level) ===
```

```
MAE : 0.0041  
RMSE: 0.005452  
R2 : 0.9568  
MAPE: 5.23%  
DA : 0.7414
```

```
[8]: y_pred_train = lgbm_model.predict(X_train)  
y_pred_val = lgbm_model.predict(X_val)  
  
train_mae, train_rmse, train_r2, train_mape, train_da = metrics_with_direction(  
    y_train.values, y_pred_train  
)  
val_mae, val_rmse, val_r2, val_mape, val_da = metrics_with_direction(  
    y_val.values, y_pred_val  
)  
  
print("=== TRAIN METRICS (level) ===")  
print(f"MAE : {train_mae:.4f}")  
print(f"RMSE: {train_rmse:.6f}")  
print(f"R2 : {train_r2:.4f}")  
print(f"MAPE: {train_mape:.2f}%")  
print(f"DA : {train_da:.4f}")  
  
print("\n=== VALIDATION METRICS (level) ===")  
print(f"MAE : {val_mae:.4f}")  
print(f"RMSE: {val_rmse:.6f}")  
print(f"R2 : {val_r2:.4f}")  
print(f"MAPE: {val_mape:.2f}%")  
print(f"DA : {val_da:.4f}")
```

```
=== TRAIN METRICS (level) ===
```

```
MAE : 0.0001  
RMSE: 0.000170  
R2 : 1.0000  
MAPE: 0.09%  
DA : 0.9154
```

```
=== VALIDATION METRICS (level) ===
```

MAE : 0.0023
RMSE: 0.003471
 R^2 : 0.9825
MAPE: 2.61%
DA : 0.8175

4 Plots

```
[11]: plt.figure(figsize=(7, 7))
plt.scatter(y_val, y_pred_val_x, alpha=0.5)

min_val = min(y_val.min(), y_pred_val_x.min())
max_val = max(y_val.max(), y_pred_val_x.max())
plt.plot([min_val, max_val], [min_val, max_val], linestyle='--')

plt.xlabel("Actual Spread")
plt.ylabel("Predicted Spread")
plt.title("Actual vs Predicted (Validation, Levels)")
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Time series: Actual vs Predicted over time ---
val_df = pd.DataFrame(
    {
        "Actual": y_val.values,
        "Predicted": y_pred_val_x,
    },
    index=val["date"]
)

plt.figure(figsize=(12, 5))
plt.plot(val_df.index, val_df["Actual"], label="Actual")
plt.plot(val_df.index, val_df["Predicted"], label="Predicted")
plt.xlabel("Time")
plt.ylabel("Spread Level")
plt.title("Actual vs Predicted Spread over Time (Validation)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Feature Importances ---
importances = xgb_model.feature_importances_

# importances = lgbm_model.feature_importances_
```

```

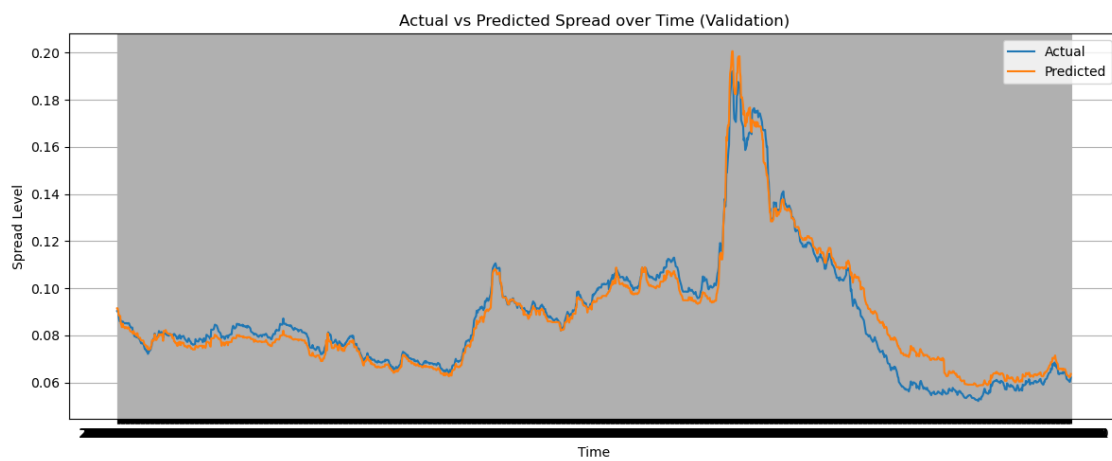
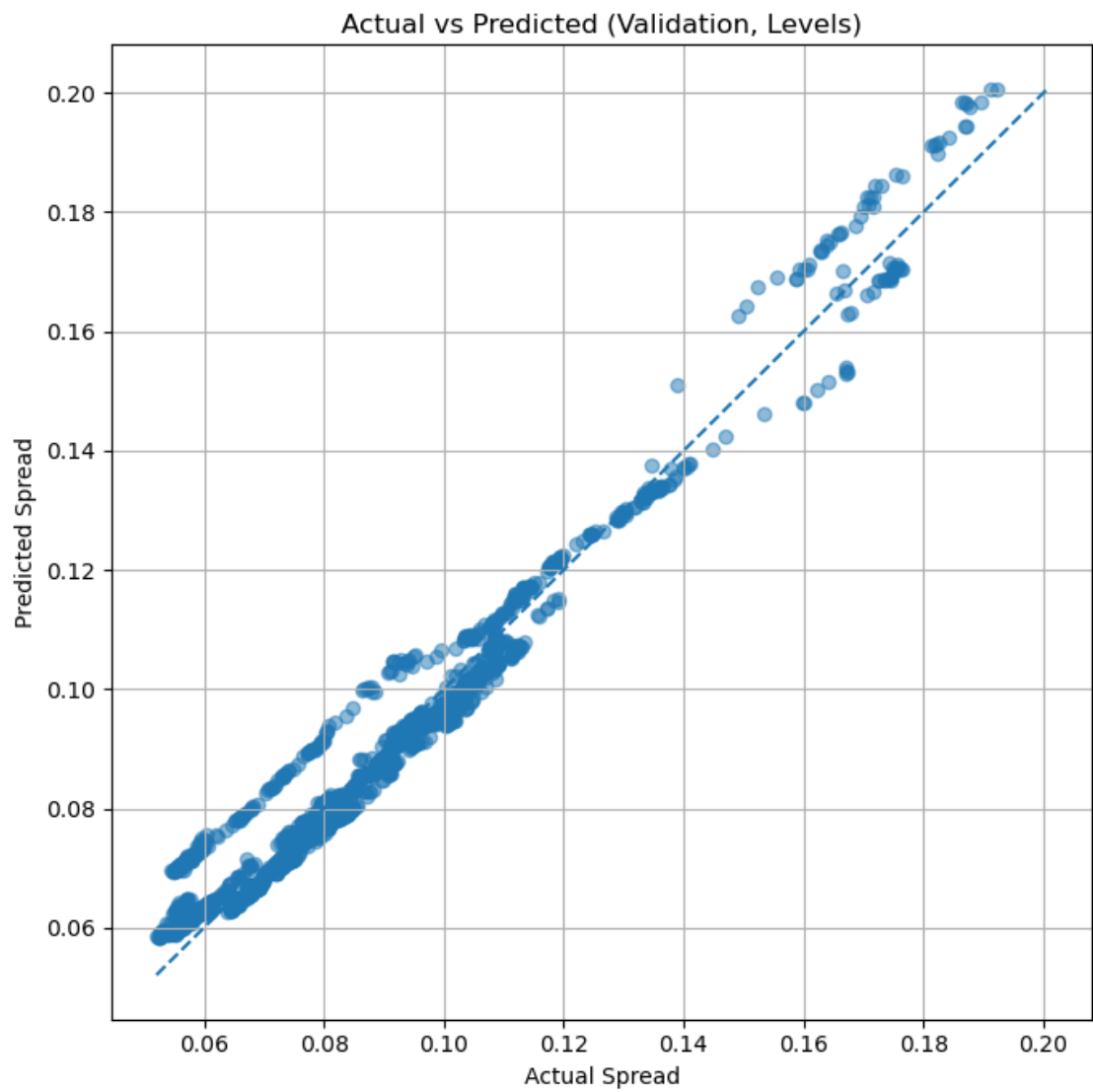
# Indices of non-zero importances
nz_idx = np.where(importances > 0.001)[0]

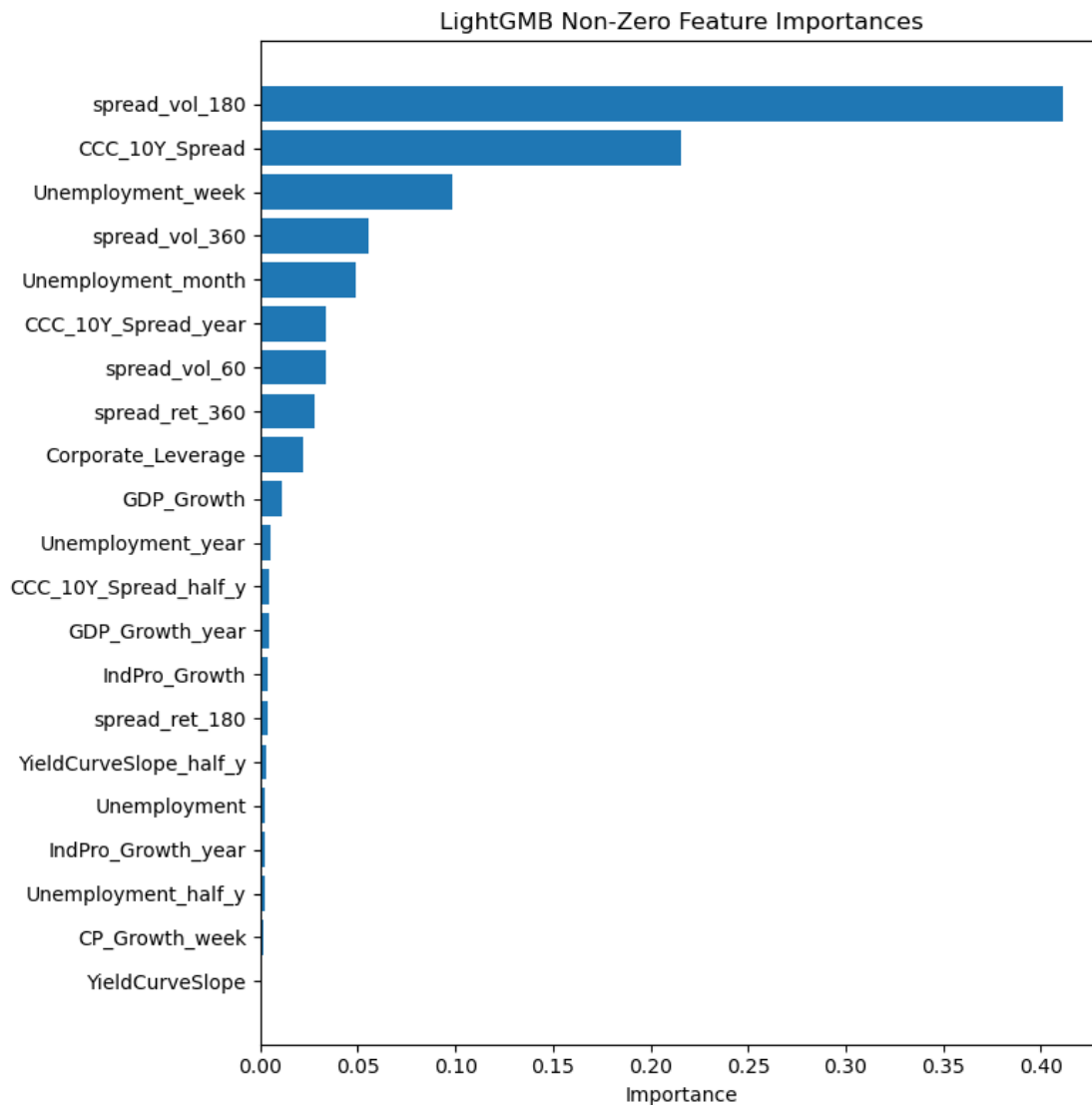
# Sorted indices (only non-zero)
sorted_nz_idx = nz_idx[np.argsort(importances[nz_idx])]

# Corresponding feature names
nz_features = np.array(feature_cols_num)[sorted_nz_idx]
nz_importances = importances[sorted_nz_idx]

plt.figure(figsize=(8, 8))
plt.barh(range(len(sorted_nz_idx)), nz_importances)
plt.yticks(range(len(sorted_nz_idx)), nz_features)
plt.xlabel("Importance")
plt.title("LightGBM Non-Zero Feature Importances")
plt.tight_layout()
plt.show()

```





```
[10]: plt.figure(figsize=(7, 7))
plt.scatter(y_val, y_pred_val, alpha=0.5)

min_val = min(y_val.min(), y_pred_val.min())
max_val = max(y_val.max(), y_pred_val.max())
plt.plot([min_val, max_val], [min_val, max_val], linestyle='--')

plt.xlabel("Actual Spread")
plt.ylabel("Predicted Spread")
plt.title("Actual vs Predicted (Validation, Levels)")
```

```

plt.grid(True)
plt.tight_layout()
plt.show()

# --- Time series: Actual vs Predicted over time ---
val_df = pd.DataFrame(
    {
        "Actual": y_val.values,
        "Predicted": y_pred_val,
    },
    index=val["date"]
)

plt.figure(figsize=(12, 5))
plt.plot(val_df.index, val_df["Actual"], label="Actual")
plt.plot(val_df.index, val_df["Predicted"], label="Predicted")
plt.xlabel("Time")
plt.ylabel("Spread Level")
plt.title("Actual vs Predicted Spread over Time (Validation)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Feature Importances ---
# importances = xgb_model.feature_importances_

importances = lgbm_model.feature_importances_

# Indices of non-zero importances
nz_idx = np.where(importances > 0.001)[0]

# Sorted indices (only non-zero)
sorted_nz_idx = nz_idx[np.argsort(importances[nz_idx])]

# Corresponding feature names
nz_features = np.array(feature_cols_num)[sorted_nz_idx]
nz_importances = importances[sorted_nz_idx]

plt.figure(figsize=(8, 8))
plt.barh(range(len(sorted_nz_idx)), nz_importances)
plt.yticks(range(len(sorted_nz_idx)), nz_features)
plt.xlabel("Importance")
plt.title("LightGMB Non-Zero Feature Importances")
plt.tight_layout()
plt.show()

```

